

GitOps avec Argo CD

Git comme source de vérité — déploiement déclaratif, sync & rollback automatiques

Formation DevOps • avec illustrations

Formateur : Haithem Mihoubi

7 modules • fil rouge : déployer nginx en GitOps • 2026

Table des matières

00	Introduction : qu'est-ce que le GitOps ?	3
01	Les principes du GitOps	6
02	Argo CD : installation & architecture	9
03	L'objet Application : déployer nginx	12
04	Synchronisation, auto-réparation & rollback	16
05	Structurer le dépôt & gérer les environnements	20
06	Secrets, bonnes pratiques & checklist	24

Introduction : qu'est-ce que le GitOps ?

1. Le problème : « qui a déployé quoi, et comment ? »

Vous avez un cluster Kubernetes. Les déploiements se font par `kubectl apply`, des scripts, parfois une modification manuelle « en urgence ». Très vite :

- **Personne ne sait** ce qui tourne réellement dans le cluster.
- Un `kubectl edit` en pleine nuit crée une **dérive** invisible : le cluster ne correspond plus à aucun fichier connu.
- **Reproduire** l'environnement (ou en recréer un après incident) relève de l'archéologie.
- Le **pipeline CI/CD** détient les **identifiants du cluster** — une cible de choix.
- Revenir à l'état d'avant un déploiement raté est **manuel** et stressant.

Le manque, c'est une **source de vérité unique** et un mécanisme qui garantit que le cluster **lui ressemble en permanence**.

2. La solution : le GitOps

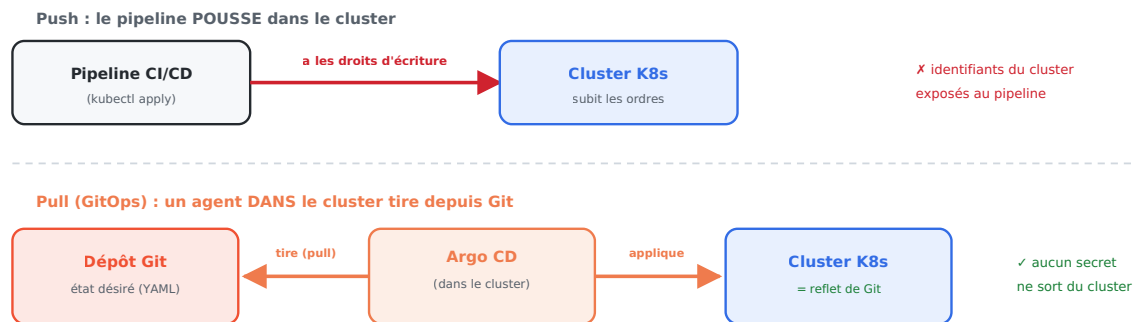
GitOps est une méthode d'exploitation où **Git est la source de vérité unique** de l'infrastructure. L'état désiré du système est **décrit dans un dépôt Git**, et un **agent dans le cluster** s'assure en continu que l'état réel **correspond** à Git.

Les quatre principes (formalisés par l'OpenGitOps) :

Principe	Signification
Déclaratif	tout l'état est décrit en YAML (pas de scripts impératifs)
Versionné & immuable	l'état vit dans Git : historisé, signé, auditable
Tiré automatiquement	un agent récupère (pull) l'état désiré depuis Git
Réconcilié en continu	l'agent corrige sans cesse les écarts cluster ↔ Git

3. Le changement de modèle : du push au pull

CI/CD classique (push) vs GitOps (pull)



En GitOps, on ne pousse plus vers le cluster : on écrit dans Git, et l'agent réconcilie le cluster avec Git.

En GitOps, on n'écrit plus vers le cluster : on écrit dans Git, et l'agent réconcilie le cluster.

	CI/CD classique (push)	GitOps (pull)
Qui applique ?	le pipeline, depuis l'extérieur	un agent dans le cluster
Secrets du cluster	exposés au pipeline	ne sortent jamais du cluster
Source de vérité	dispersée (scripts, mémoire)	Git , unique
Dérive manuelle	non détectée	détectée et corrigée
Rollback	manuel	<code>git revert</code>

4. Argo CD : l'agent GitOps

Ce cours utilise **Argo CD**, l'outil GitOps le plus répandu pour Kubernetes (projet CNCF). Argo CD tourne **dans** le cluster, surveille un dépôt Git, et **synchronise** les manifestes qu'il y trouve vers le cluster.

Alternative notable : **Flux**. Les principes sont identiques ; Argo CD se distingue par une **interface graphique** riche qui rend le concept très visuel — idéal pour apprendre.

5. Ce que le GitOps apporte concrètement

Bénéfice	Pourquoi
Traçabilité totale	chaque changement = un commit (qui, quoi, quand, pourquoi)
Rollback trivial	revenir en arrière = <code>git revert</code>
Pas de dérive	l'agent réaligne le cluster sur Git automatiquement
Reproductibilité	recréer un cluster = pointer Argo CD sur le dépôt
Sécurité	les identifiants du cluster restent dans le cluster
Revue de code	un déploiement passe par une Pull Request

6. Notre fil rouge : déployer nginx en GitOps

Comme dans les cours précédents, **nginx** sert d'exemple. On va :

1. comprendre les **principes** GitOps et la boucle de réconciliation ;
2. installer **Argo CD** et découvrir son architecture ;
3. créer une **Application** Argo CD qui déploie nginx depuis Git ;
4. maîtriser la **synchronisation**, l'**auto-réparation** et le **rollback** ;
5. structurer un **dépôt** multi-environnements (dev/staging/prod) ;
6. appliquer les **bonnes pratiques** (secrets, App of Apps, sécurité).

7. Où le GitOps s'inscrit dans la chaîne

Le GitOps **complète** ce que vous connaissez déjà :

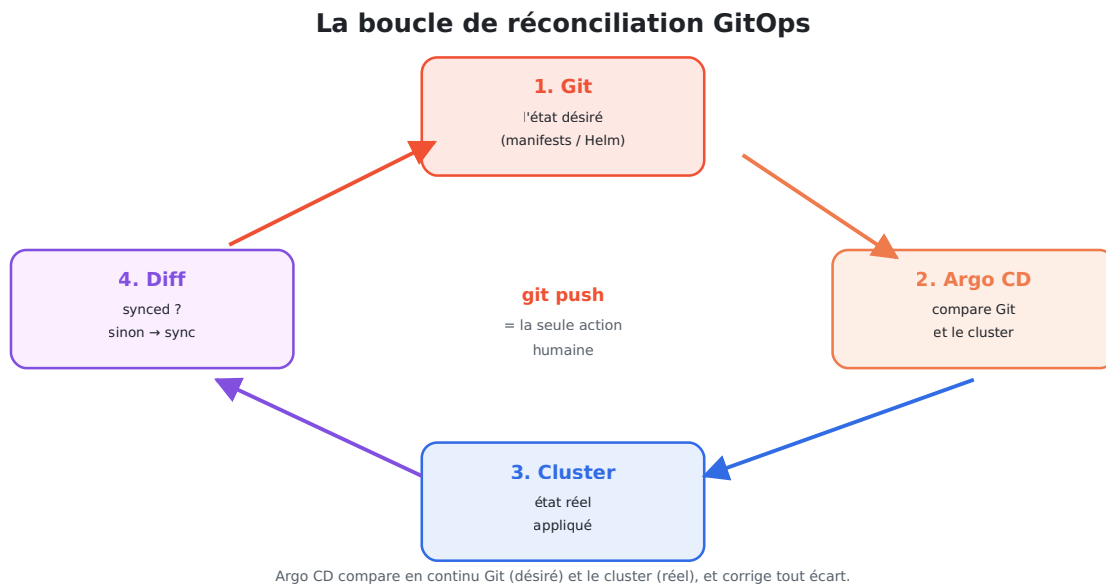
- **CI** (GitHub Actions) construit l'image et la pousse sur un registry, puis **met à jour un manifeste dans Git** (le nouveau tag d'image).
- **GitOps** (Argo CD) détecte ce changement dans Git et **déploie** sur le cluster.

La frontière est nette : la **CI** s'arrête à Git ; le **CD** (GitOps) part de Git. Le pipeline n'a **plus besoin** d'accéder au cluster.

Pré-requis : Kubernetes (Pods, Deployments, Services) et des bases de Git. Helm est un plus (Argo CD sait déployer des charts). Place aux principes, en détail.

Les principes du GitOps

Avant l'outil, la **méthode**. Le GitOps tient en une idée — Git fait foi — et une mécanique — la **boucle de réconciliation**.



Argo CD compare en continu Git (désiré) et le cluster (réel), et corrige tout écart.

1. Git, source de vérité unique

En GitOps, **rien** ne se déploie en dehors de Git. L'état complet du système — Deployments, Services, ConfigMaps, Ingress... — est décrit dans un **dépôt Git**.

Conséquence	Bénéfice
Tout changement passe par un commit	historique complet, qui/quoi/quand
Un déploiement = une Pull Request	revue, validation, discussion
L'état est versionné	on peut comparer, revenir, auditer
Le dépôt décrit l'infra	recréer un cluster = repartir du dépôt

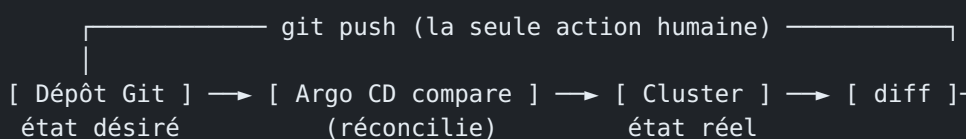
La règle d'or : si ce n'est pas dans Git, ça ne devrait pas être dans le cluster. Le dépôt est la **seule** interface de déploiement.

2. La boucle de réconciliation

C'est le cœur du GitOps. Un agent (Argo CD) exécute **en permanence** :

1. **Observer Git** : quel est l'état **désiré** ? (les YAML du dépôt)
2. **Observer le cluster** : quel est l'état **réel** ? (les objets en place)
3. **Comparer** (diff) : y a-t-il un écart ?
4. **Réconcilier** : si écart, **appliquer Git** pour réaligner le cluster.

Puis on recommence, indéfiniment. C'est le même principe que les **contrôleurs** Kubernetes (état désiré → état réel), mais étendu à **Git**.



3. Déclaratif, pas impératif

Impératif (à éviter)	Déclaratif (GitOps)
<code>kubectl scale deploy/nginx --replicas=5</code>	écrire <code>replicas: 5</code> dans le YAML, committer
<code>kubectl set image ...</code>	modifier le tag d'image dans Git
une commande, un effet ponctuel	un état que l'agent maintient

On ne décrit plus **les étapes** (« fais ceci, puis cela »), mais **le résultat voulu** (« voici à quoi le système doit ressembler »). L'agent se charge du **comment**.

4. La fin de la dérive de configuration

La **dérive** (configuration drift), c'est quand le cluster s'éloigne de ce qui est écrit : un `kubectl edit` d'urgence, une modification « temporaire » oubliée...

En GitOps, l'agent **détecte** cette dérive (le cluster $\neq$ Git) et peut la **corriger automatiquement** (auto-réparation). Le cluster **converge toujours** vers Git.

Exemple : quelqu'un passe nginx à 10 réplicas avec `kubectl scale`. Argo CD voit que Git dit 3, marque l'application **OutOfSync**, et (si l'auto-heal est actif) **revient à 3**. Pour vraiment passer à 10, il faut **modifier Git**.

5. Les deux dépôts : application vs configuration

Une bonne pratique GitOps **sépare** le code applicatif de la configuration de déploiement :

Dépôt	Contient	Modifié par
dépôt applicatif	le code source, le Dockerfile	les développeurs
dépôt de config (GitOps)	les manifestes K8s / Helm values	la CI (tag d'image) + les ops

La CI construit l'image, la pousse, puis **met à jour le tag** dans le dépôt de config. Argo CD, qui surveille ce dépôt, déploie la nouvelle version. Les responsabilités sont nettes.

6. Ce que le GitOps n'est PAS

- **≠ « juste mettre ses YAML sur GitHub ».** Sans agent de réconciliation, il n'y a pas de garantie que le cluster suive Git.
- **≠ un simple `kubectl apply` dans un pipeline.** Le push reste impératif et ponctuel ; le GitOps **observe et corrige en continu**.
- **≠ réservé aux gros clusters.** Le bénéfice (traçabilité, rollback, pas de dérive) vaut dès le premier environnement.

7. Récapitulatif

Principe	En une phrase
Source de vérité	Git décrit tout l'état désiré
Déclaratif	on décrit le résultat , pas les étapes
Pull	un agent dans le cluster tire depuis Git
Réconciliation	l'agent maintient cluster = Git en continu
Anti-dérive	toute modification hors Git est détectée (et corrigée)

À retenir : GitOps = Git comme unique source de vérité + une boucle de réconciliation qui aligne sans cesse le cluster sur Git. Le module suivant met cela en œuvre avec **Argo CD**.

Argo CD : installation & architecture

Argo CD est l'agent qui met en œuvre le GitOps sur Kubernetes. Il tourne **dans** le cluster, surveille un dépôt Git et synchronise les manifestes.

1. Installer Argo CD

Argo CD s'installe... de façon déclarative, dans son propre namespace :

```
kubectl create namespace argocd
kubectl apply -n argocd -f \
  https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
```

Accéder à l'interface web :

```
kubectl port-forward svc/argocd-server -n argocd 8080:443
# → https://localhost:8080

# Mot de passe initial de l'admin
kubectl -n argocd get secret argocd-initial-admin-secret \
  -o jsonpath="{.data.password}" | base64 -d
```

Et la CLI :

```
argocd login localhost:8080      # se connecter
argocd app list                 # lister les applications
```

2. Les composants d'Argo CD

Argo CD se compose de quelques services, chacun avec un rôle précis :

Composant	Rôle
API Server	l'interface (UI web, CLI, API) ; gère l'authentification
Repository Server	clone les dépôts Git et génère les manifestes (Helm, Kustomize...)
Application Controller	le cœur : compare Git ↔ cluster et réconcilie
Redis	cache interne
Dex (optionnel)	intégration SSO (OIDC, LDAP...)

L'Application Controller est l'élément central : c'est lui qui exécute la boucle de réconciliation pour chaque application, en continu.

3. Ce qu'Argo CD sait déployer

Argo CD ne se limite pas aux YAML bruts. Le **Repository Server** sait **rendre** plusieurs formats avant d'appliquer :

Format	Argo CD...
Manifestes K8s	applique les YAML tels quels
Helm	exécute <code>helm template</code> (avec vos values) puis applique
Kustomize	exécute <code>kustomize build</code> puis applique
Jsonnet / plugins	via des générateurs personnalisés

Vous pouvez donc faire du GitOps **sur vos charts Helm** : Argo CD pointe le chart dans Git, applique vos `values`, et maintient la release synchronisée. Helm + GitOps se marient parfaitement.

4. L'interface graphique : le GitOps visuel

L'UI d'Argo CD est un de ses atouts. Pour chaque application, elle affiche :

- l'**arbre des ressources** (Application → Deployment → ReplicaSet → Pods...) ;
- l'état de **synchronisation** (`Synced` / `OutOfSync`) ;
- l'état de **santé** (`Healthy` / `Degraded` / `Progressing`) ;
- le **diff** exact entre Git et le cluster ;
- l'historique des syncs, avec **rollback** en un clic.

C'est extrêmement pédagogique : on **voit** la dérive apparaître en orange, puis disparaître après la synchronisation. Le concept GitOps devient concret.

5. Multi-cluster : un Argo CD, plusieurs clusters

Argo CD peut gérer **plusieurs clusters** depuis une seule instance :

```
argocd cluster add mon-cluster-prod      # enregistrer un cluster cible
argocd cluster list
```

→ une instance centrale d'Argo CD déploie sur le cluster `dev`, `staging` **et** `prod`, chacun étant une **destination** différente (voir le module suivant).

6. Sécurité et accès (RBAC)

Argo CD gère ses propres droits, distincts de ceux du cluster :

- **Projects** : regroupent des applications et **restreignent** ce qu'elles peuvent faire (quels dépôts, quels clusters, quels namespaces).
- **RBAC** : qui peut voir/synchroniser/supprimer quelles applications.
- **SSO** : connexion via le fournisseur d'identité de l'entreprise (OIDC, GitHub...).

```
apiVersion: argoproj.io/v1alpha1
kind: AppProject
metadata:
  name: quickbite
  namespace: argocd
spec:
  sourceRepos:
    - 'https://github.com/mon-orga/quickbite-config' # dépôts autorisés
  destinations:
    - server: https://kubernetes.default.svc
      namespace: 'prod' # namespaces autorisés
```

7. Vue d'ensemble

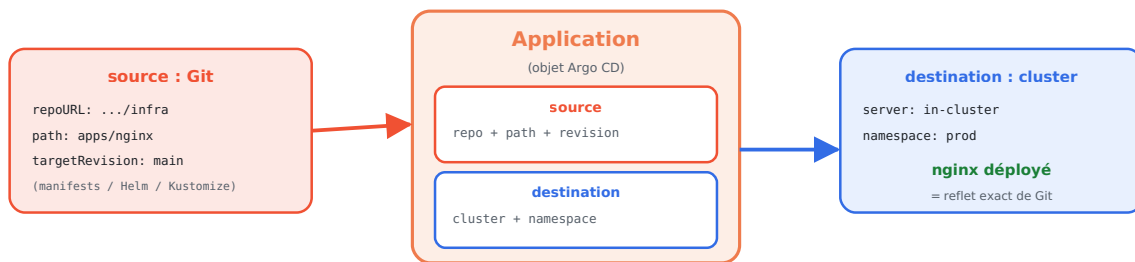
```
[ Dépôt Git ] --(clone)--> [ Repository Server ] --(rend les manifests)-->
[ UI / CLI ] --> [ API Server ] [ Application Controller ] --(applique)--> [ Cluster ]
                                   (compare & réconcilie en boucle)
```

À retenir : Argo CD s'installe dans le cluster, son **Application Controller** exécute la boucle de réconciliation, le **Repository Server** rend les manifests (YAML/Helm/Kustomize), et l'**UI** rend tout visible. Reste à lui dire **quoi** déployer : l'objet **Application**.

L'objet Application : déployer nginx

L'**Application** est l'objet central d'Argo CD. Il fait le lien entre **un chemin dans Git** (l'état désiré) et **un cluster + namespace** (où déployer).

L'objet Application : le lien Git → Cluster



Une Application dit : « prends CE chemin de CE dépôt et applique-le sur CE cluster ».

Une Application dit : « prends CE chemin de CE dépôt et applique-le sur CE cluster ».

1. Anatomie d'une Application

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: nginx
  namespace: argocd
spec:
  project: default

  source:
    repoURL: https://github.com/mon-orga/quickbite-config
    path: apps/nginx
    targetRevision: main

  destination:
    server: https://kubernetes.default.svc
    namespace: prod

  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
```

— D'OÙ vient l'état désiré

le dossier dans le dépôt

branche, tag ou commit

— OÙ déployer

le cluster (ici, local)

— COMMENT synchroniser

supprimer ce qui disparaît de Git

corriger les dérives manuelles

créer le namespace si absent

2. Les trois blocs essentiels

Bloc	Répond à	Champs clés
<code>source</code>	D'où ?	<code>repoURL</code> , <code>path</code> , <code>targetRevision</code>
<code>destination</code>	Où ?	<code>server</code> (cluster), <code>namespace</code>
<code>syncPolicy</code>	Comment ?	<code>automated</code> , <code>prune</code> , <code>selfHeal</code>

`targetRevision` mérite attention : `main` suit la branche (déploie chaque commit) ; un **tag** (`v1.2.0`) ou un **commit** fige une version précise — utile pour la prod.

3. Créer l'Application

Trois façons, pour le même résultat.

En YAML (recommandé — c'est du GitOps jusqu'au bout)

```
kubectl apply -f nginx-application.yaml
```

En CLI

```
argocd app create nginx \
  --repo https://github.com/mon-orga/quickbite-config \
  --path apps/nginx \
  --dest-server https://kubernetes.default.svc \
  --dest-namespace prod \
  --sync-policy automated
```

Dans l'UI

+ **New App**, remplir source/destination/sync, **Create**.

4. Le dossier `apps/nginx` dans Git

L'Application pointe `path: apps/nginx`. Ce dossier contient l'état désiré de nginx — de simples manifestes :

```

quickbite-config/
├─ apps/
│   └─ nginx/
│       ├── deployment.yaml    # le Deployment nginx (3 réplicas)
│       ├── service.yaml       # le Service
│       └─ ingress.yaml        # l'Ingress

```

...ou un **chart Helm** :

```

spec:
  source:
    repoURL: https://github.com/mon-orga/quickbite-config
    path: charts/nginx
    helm:
      valueFiles:
        - values-prod.yaml      # Argo CD exécute helm template avec ces values

```

5. Suivre l'état de l'Application

```

argocd app get nginx          # état sync + santé + ressources
argocd app list               # toutes les applications
argocd app resources nginx    # l'arbre des objets gérés

```

Deux états indépendants, à bien distinguer :

État	Question	Valeurs
Sync	le cluster correspond-il à Git ?	Synced / OutOfSync
Health	les ressources fonctionnent-elles ?	Healthy / Progressing / Degraded

Une application peut être **Synced mais Degraded** (le cluster reflète Git, mais un Pod nginx plante) — ou **OutOfSync mais Healthy** (ça tourne, mais une dérive existe). Les deux axes sont complémentaires.

6. Le déploiement, désormais, c'est un commit

Une fois l'Application en place avec [automated](#) , **modifier nginx = modifier Git** :

```

# Passer nginx de 3 à 5 réplicas
# → on édite apps/nginx/deployment.yaml (replicas: 5) dans Git
git add apps/nginx/deployment.yaml
git commit -m "nginx: 3 → 5 réplicas"
git push
# → Argo CD détecte le changement et applique automatiquement. Aucun kubectl.

```

C'est tout le GitOps en une phrase : **on déploie en poussant dans Git.**

À retenir : L'Application relie `source` (un chemin Git) à `destination` (un cluster/namespace), avec une `syncPolicy`. Les manifestes peuvent être bruts, Helm ou Kustomize. Une fois en place, déployer = committer. Le module suivant détaille la **synchronisation**.

Synchronisation, auto-réparation & rollback

Le cœur opérationnel d'Argo CD : comment il **synchronise** le cluster avec Git, **corrige** les dérives et permet de **revenir en arrière**.

États de synchronisation & auto-réparation



Scénario : quelqu'un modifie le cluster à la main (kubectl edit)



Avec selfHeal activé, toute modification hors Git est automatiquement annulée : Git fait foi.

Une dérive manuelle est détectée (OutOfSync) puis annulée automatiquement (selfHeal) : Git fait foi.

1. Les états de synchronisation

État	Signification
Synced	le cluster correspond exactement à Git
OutOfSync	un écart existe (Git a changé, ou le cluster a dérivé)

Argo CD recalcule cet état **en continu** (par défaut toutes les ~3 min, ou immédiatement via un **webhook** Git).

2. Sync manuelle vs automatique

Synchronisation manuelle

Sans politique automatique, Argo CD **détecte** les écarts mais **n'applique pas** seul : un humain (ou la CI) déclenche la synchro.

```
argocd app sync nginx      # appliquer maintenant l'état de Git
```

→ pratique pour la **production**, où l'on veut **valider** avant de déployer.

Synchronisation automatique

```
syncPolicy:
  automated:
    prune: true
    selfHeal: true
```

→ Argo CD applique **automatiquement** dès que Git change. Idéal pour `dev` / `staging`.

3. `prune` : supprimer ce qui disparaît de Git

Par défaut, Argo CD **n'efface pas** un objet retiré de Git (par prudence). Avec `prune: true`, supprimer un fichier du dépôt **supprime** l'objet correspondant dans le cluster.

Exemple : vous retirez `ingress.yaml` du dossier `apps/nginx`. Avec `prune`, l'Ingress est supprimé du cluster à la prochaine synchro. Sans `prune`, il resterait orphelin.

4. `selfHeal` : annuler les dérives manuelles

C'est la fonctionnalité la plus spectaculaire. Avec `selfHeal: true`, **toute** modification faite directement sur le cluster est **annulée** : Argo CD réapplique Git.

```
# Quelqu'un bricole le cluster à la main...
kubectl scale deployment nginx --replicas=10 -n prod

# ...Argo CD le voit (OutOfSync), et puisque Git dit 3, il revient à 3.
# La dérive est annulée. Pour passer à 10, il FAUT modifier Git.
```

`selfHeal` rend le cluster immuable depuis l'extérieur de Git. C'est la garantie ultime contre la dérive : le seul moyen de changer quelque chose, c'est un commit.

5. Les hooks de synchronisation

Argo CD sait exécuter des actions **autour** d'une synchro (migrations de base, tests...) grâce à des **annotations** sur des ressources (souvent des Jobs) :

Hook	Quand
PreSync	avant d'appliquer (ex. migration de schéma)
Sync	pendant la synchro
PostSync	après (ex. tests de fumée, notification)
SyncFail	si la synchro échoue

```
metadata:
  annotations:
    argocd.argoproj.io/hook: PreSync
```

6. Le rollback : revenir à une version précédente

Tout l'historique des synchros est conservé. Deux façons de revenir en arrière.

La façon GitOps (recommandée)

```
git revert <commit-fautif>
git push
# → Argo CD re-synchronise vers l'état d'avant. L'historique reste propre et auditable.
```

La façon Argo CD (dépannage rapide)

```
argocd app history nginx          # lister les déploiements passés
argocd app rollback nginx 42      # revenir à la révision applicative 42
```

Préférez `git revert` : il garde Git comme **seule** source de vérité. Un `argocd rollback` met temporairement le cluster en avance sur Git (donc **OutOfSync**) ; à n'utiliser qu'en urgence, en réalignant Git ensuite.

7. Déclencher la synchro instantanément (webhook)

Plutôt que d'attendre le prochain cycle, on configure un **webhook** Git (GitHub, GitLab...) qui prévient Argo CD à chaque push :

```
Dépôt Git ▶ Settings ▶ Webhooks ▶ https://argocd.exemple.com/api/webhook
```

→ le déploiement devient **quasi instantané** après le `git push`.

8. Récapitulatif des comportements

Politique	Effet
<code>automated</code>	applique automatiquement les changements de Git
<code>prune: true</code>	supprime les objets retirés de Git
<code>selfHeal: true</code>	annule les modifications faites hors Git
webhook	synchronise dès le <code>git push</code> (sinon ~3 min)

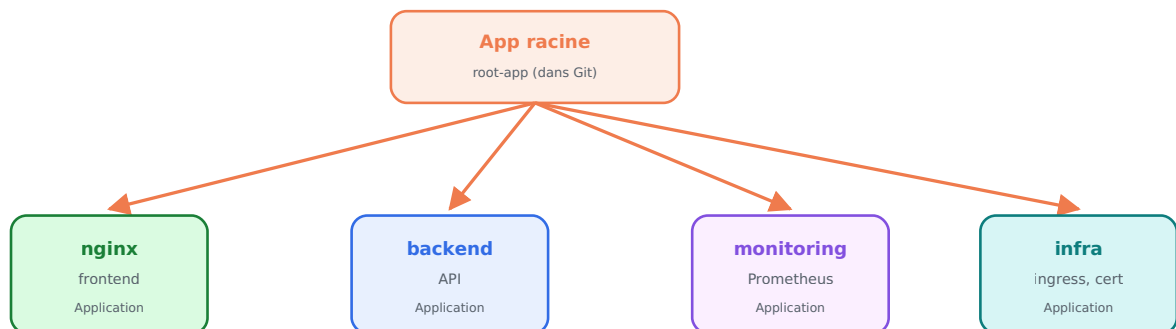
À retenir : `Synced` / `OutOfSync` mesurent l'écart avec Git ; `automated` , `prune` et `selfHeal` automatisent l'alignement ; le rollback se fait idéalement par `git revert` . Git reste, en toutes circonstances, **la** vérité.

Structurer le dépôt & gérer les environnements

À l'échelle, on gère **plusieurs applications** et **plusieurs environnements** (dev/staging/prod). L'organisation du dépôt Git et le motif **App of Apps** rendent cela maintenable.

Organiser à l'échelle : le motif App of Apps

Une Application racine qui référence d'autres Applications



On gère des dizaines d'apps et plusieurs environnements (dev/staging/prod) de façon déclarative, depuis Git.

Une Application racine référence d'autres Applications : on gère des dizaines d'apps depuis Git.

1. Organiser le dépôt de configuration

Une structure claire et lisible, par application et par environnement :

```

quickbite-config/
├── apps/
│   ├── nginx/
│   │   ├── base/                # commun à tous les environnements
│   │   │   ├── deployment.yaml
│   │   │   └── service.yaml
│   │   └── overlays/            # variations par environnement (Kustomize)
│   │       ├── dev/
│   │       ├── staging/
│   │       └── prod/
│   ├── backend/
│   └── monitoring/
├── argocd/
│   ├── root-app.yaml            # l'Application racine (App of Apps)
│   └── applications/            # une Application par app/env
│       ├── nginx-dev.yaml
│       ├── nginx-prod.yaml
│       └── ...
  
```

2. Gérer les environnements : base + overlays

On **ne duplique pas** les manifestes par environnement. On garde une **base** commune et des **surcharges** (overlays) qui ne changent que le nécessaire.

```
# apps/nginx/overlays/prod/kustomization.yaml
resources:
- ../../base
patches:
- patch: |-
  - op: replace
    path: /spec/replicas
    value: 5                                # prod : 5 réplicas (base : 1)
```

Environnement	Différences typiques
dev	1 réplica, image <code>:latest</code> , ressources minimales
staging	proche de la prod, image taguée
prod	N réplicas, image figée par tag, limites strictes, sync manuelle

Avec **Helm**, le même principe s'applique : un chart, un fichier `values-<env>.yaml` par environnement. Argo CD pointe le bon fichier de values dans chaque Application.

3. Le motif App of Apps

Créer chaque Application à la main ne passe pas à l'échelle. Le motif **App of Apps** : une Application **racine** qui déploie... **d'autres** Applications.

```
# argocd/root-app.yaml - l'Application racine
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: root
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/mon-orga/quickbite-config
    path: argocd/applications          # un dossier PLEIN d'Applications
    targetRevision: main
  destination:
    server: https://kubernetes.default.svc
    namespace: argocd
  syncPolicy:
    automated: { prune: true, selfHeal: true }
```

Le dossier `argocd/applications/` contient une Application par app/env. On installe **une seule fois** la racine ; ensuite, **ajouter une app = ajouter un fichier YAML** dans Git.

```
kubectl apply -f argocd/root-app.yaml      # une fois
# → la racine déploie nginx-dev, nginx-prod, backend, monitoring...
```

4. ApplicationSet : générer des Applications

Pour des cas très répétitifs (la **même** app sur N environnements ou N clusters), Argo CD propose l'**ApplicationSet** : il **génère** les Applications à partir d'un patron.

```
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
  name: nginx
  namespace: argocd
spec:
  generators:
    - list:
        elements:
          - env: dev
          - env: staging
          - env: prod
  template:
    metadata:
      name: 'nginx-{{env}}'
    spec:
      source:
        repoURL: https://github.com/mon-orga/quickbite-config
        path: 'apps/nginx/overlays/{{env}}'
        targetRevision: main
      destination:
        server: https://kubernetes.default.svc
        namespace: '{{env}}'
      syncPolicy:
        automated: { prune: true, selfHeal: true }
```

→ une seule définition crée **nginx-dev**, **nginx-staging** et **nginx-prod**. Ajouter un environnement = ajouter une ligne.

5. Promouvoir une version d'un environnement à l'autre

En GitOps, **promouvoir** = un changement dans Git :

1. La CI déploie l'image `nginx:1.28` et met à jour `overlays/dev` → Argo déploie en dev.
2. Une fois validé, on **reporte** le tag `1.28` dans `overlays/staging` (PR) → staging.
3. Puis dans `overlays/prod` (PR avec revue) → prod.

Chaque promotion est une **Pull Request** : revue, traçabilité, et rollback par `git revert`. La « pipeline de promotion » vit entièrement dans Git.

6. Bonnes pratiques de structure

- **Séparer** le dépôt applicatif (code) du dépôt de config (manifestes).
- **Une base + des overlays** (ou un chart + des values) — jamais de copier-coller par env.
- **App of Apps** ou **ApplicationSet** pour ne pas créer les Applications à la main.
- **Sync manuelle en prod**, automatique en dev/staging.
- **Tag d'image figé** en prod (pas `latest`) pour des déploiements déterministes.

À retenir : un dépôt bien rangé (base/overlays), le motif **App of Apps** (ou **ApplicationSet**) et la promotion par Pull Request permettent de gérer des dizaines d'applications sur plusieurs environnements — **tout depuis Git**.

Secrets, bonnes pratiques & checklist

Le GitOps soulève une question évidente : **comment gérer les secrets** si tout est dans Git ? Plus les pratiques qui font un GitOps sain, et une checklist finale.

1. Le défi des secrets en GitOps

On ne peut **jamais** committer un mot de passe ou une clé **en clair** dans Git : le dépôt est historisé, cloné, parfois public. Or le GitOps veut **tout** dans Git. Solution : committer les secrets **chiffrés**.

Approche	Principe
Sealed Secrets (Bitnami)	on committe un secret chiffré ; un contrôleur le déchiffre dans le cluster
SOPS (+ age/KMS)	chiffre les valeurs dans le YAML ; déchiffré au déploiement
External Secrets Operator	le secret reste dans un coffre externe (Vault, AWS/GCP) ; l'opérateur le synchronise

Exemple : Sealed Secrets

```
# Chiffrer un secret classique en "SealedSecret" (sûr à committer)
kubeseal -f secret.yaml -w sealed-secret.yaml
git add sealed-secret.yaml && git commit -m "secret nginx-tls (chiffré)"
# → dans le cluster, le controller le déchiffre en vrai Secret
```

La règle : ce qui est dans Git doit être **chiffré** ou n'être qu'une **référence** vers un coffre. Jamais de secret en clair, même dans un dépôt privé.

2. Sécuriser Argo CD

- **RBAC + Projects** : limiter chaque application aux dépôts, clusters et namespaces autorisés (principe du moindre privilège).
- **SSO** : connecter Argo CD au fournisseur d'identité de l'entreprise (OIDC, GitHub).
- **Désactiver l'admin** local une fois le SSO en place.
- **Protéger la branche** déployée (`main`) : Pull Requests obligatoires, revue, statut CI.
- **Signer les commits** pour garantir l'origine des changements.

3. Organisation et qualité

- **Deux dépôts** : code applicatif d'un côté, configuration GitOps de l'autre.
- **Base + overlays** (ou chart + values) : zéro duplication entre environnements.
- **App of Apps / ApplicationSet** : ne pas créer les Applications à la main.
- **Tag d'image figé en prod** (jamais `latest`) : déploiements déterministes et rollback fiable.
- **Sync manuelle en prod**, automatique en dev/staging.

4. Fiabilité du déploiement

- `selfHeal` pour empêcher toute dérive (sauf raison précise de l'éviter).
- `prune` pour que la suppression dans Git supprime réellement dans le cluster.
- **Hooks** `PreSync` pour les migrations de base de données.
- **Webhook Git** pour des synchronos quasi instantanées.
- **Notifications** (Slack, e-mail) sur les échecs de synchro (`argocd-notifications`).

5. CI et GitOps : qui fait quoi

Frontière nette entre **intégration** (CI) et **déploiement** (GitOps) :

Étape	Outil	Action
1. Build & test	GitHub Actions	construit l'image, la pousse sur le registry
2. Mise à jour de config	GitHub Actions	met à jour le tag d'image dans le dépôt GitOps (commit/PR)
3. Déploiement	Argo CD	détecte le commit et synchronise le cluster

```
# Étape 2 dans un workflow CI : bump du tag, puis push (déclenche Argo CD)
- name: Mettre à jour le tag d'image dans le dépôt GitOps
  run: |
    yq -i '.image.tag = "${{ github.sha }}"' apps/nginx/overlays/prod/values.yaml
    git commit -am "nginx → ${{ github.sha }}" && git push
```

Le pipeline n'a plus accès au cluster. Il s'arrête à Git ; Argo CD prend le relais.
C'est plus simple **et** plus sûr.

6. Dépannage : pannes courantes

Symptôme	Cause probable	Solution
Reste <code>OutOfSync</code>	sync auto désactivée, ou erreur de manifeste	<code>argocd app sync</code> , lire le diff
<code>Degraded</code>	un Pod plante	<code>kubectl logs</code> , <code>argocd app get</code>
Dérive qui revient sans cesse	un autre outil modifie le cluster	identifier l'acteur, ou désactiver <code>selfHeal</code> ciblé
Secret en clair commité	mauvaise pratique	révoquer le secret, passer à Sealed Secrets/SOPS
<code>ComparisonError</code>	dépôt inaccessible / mauvais <code>path</code>	vérifier <code>repoURL</code> , droits, chemin
Helm ne rend pas	mauvais <code>valueFiles</code>	vérifier le chemin des values dans la source

7. Checklist d'un GitOps sain

- [] Git est la **source de vérité unique** ; aucun `kubectl apply` hors GitOps.
- [] Dépôt **applicatif** et dépôt **de config** séparés.
- [] **Aucun secret en clair** : Sealed Secrets / SOPS / External Secrets.
- [] **base + overlays** (ou chart + values) par environnement.
- [] **App of Apps** ou **ApplicationSet** pour la mise à l'échelle.
- [] `selfHeal` + `prune` activés (sauf exception justifiée).
- [] **Sync manuelle en prod**, automatique ailleurs ; tags d'image **figés** en prod.
- [] **RBAC/Projects** restrictifs ; **SSO** ; branche `main` protégée.
- [] **Webhook** Git pour des synchronos rapides ; **notifications** sur échec.
- [] Rollback maîtrisé par `git revert`.

Conclusion

Avec **nginx** comme fil rouge, vous avez mis en place un déploiement **GitOps** complet : Git comme **source de vérité**, **Argo CD** comme agent de réconciliation, l'objet **Application** comme lien Git→cluster, la **synchronisation** avec auto-réparation, une structure **multi-environnements** (App of Apps / ApplicationSet), et une gestion **sûre** des secrets.

Le déploiement n'est plus une action risquée : c'est un **commit revu en Pull Request**, que le cluster reflète automatiquement et durablement. Combiné à la CI (GitHub Actions), à l'orchestration (Kubernetes), au packaging (Helm) et au monitoring (Prometheus/Grafana), le GitOps **boucle** une chaîne DevOps moderne et professionnelle.

Pour aller plus loin : Argo Rollouts (canary & blue-green déclaratifs), **Argo Workflows** (pipelines dans le cluster), les **stratégies de promotion** multi- clusters, et la **policy as code** (OPA/Kyverno) pour valider les manifestes avant déploiement.